

Búsqueda Local (1a. parte)

Hill Climbing

Javier Béjar, Javier Vázquez

Algoritmos Básicos de la Inteligencia Artificial - 2022/2023 1Q

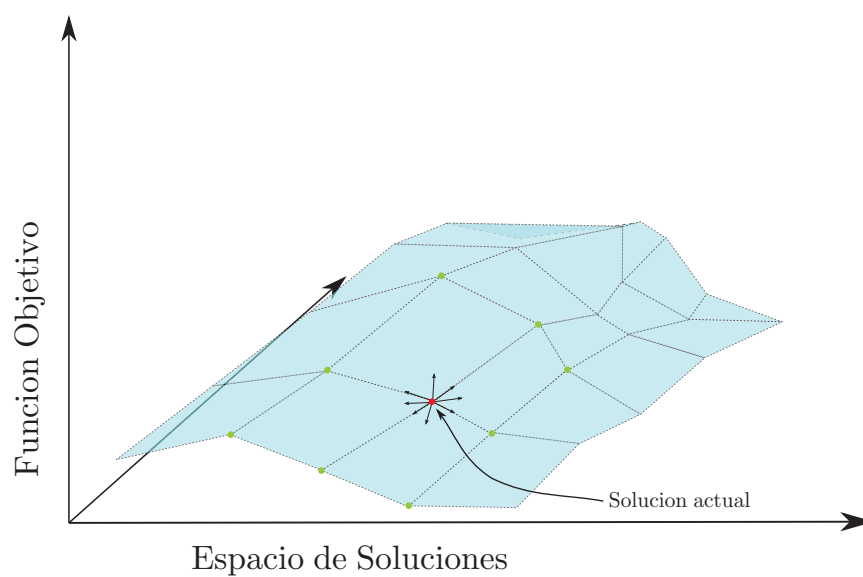
CS - GIA



Introducción

- ⦿ A veces el camino para llegar a la solución no nos importa, buscamos en el espacio de soluciones
- ⦿ Queremos la mejor de entre las soluciones posibles alcanzable en un tiempo razonable (el óptimo es imposible)
- ⦿ Tenemos una función que nos evalúa la calidad de la solución, pero que no está ligada a ningún coste necesariamente
- ⦿ La búsqueda se realiza desde una solución inicial que intentamos mejorar modificándola (operadores)
- ⦿ Los operadores nos mueven entre soluciones vecinas

2



3

- ⊙ La función heurística:
 - Aproxima la calidad de una solución (no representa un coste)
 - Hemos de optimizarla (maximizarla o minimizarla)
 - Combinará los elementos del problema y sus restricciones (posiblemente con diferentes pesos)
 - No hay ninguna restricción sobre como ha de ser la función, solo ha de representar las relaciones de calidad entre las soluciones
 - Puede tomar valores positivos o negativos

- ⊙ El tamaño del espacio de soluciones por lo general no permite obtener el óptimo
- ⊙ Los algoritmos no pueden hacer una exploración sistemática
- ⊙ La función heurística se usará para podar el espacio de búsqueda (soluciones que no merece la pena explorar)
- ⊙ No se suele guardar historia del camino recorrido (el gasto de memoria es mínimo)
- ⊙ La falta total de memoria puede suponer un problema (bucles)

Hill Climbing

Escalada (Hill climbing)

- ⦿ Escalada simple
 - Se busca cualquier operación que suponga una mejora respecto al padre
- ⦿ Escalada por máxima pendiente (steepest-ascent hill climbing, gradient search)
 - Se selecciona el mejor movimiento (no el primero de ellos) que suponga mejora respecto al estado actual

Algoritmo: Hill Climbing

Actual $\leftarrow$ Estado_inicial

fin $\leftarrow$ falso

mientras no fin **hacer**

 Hijos $\leftarrow$ generar_sucesores(Actual)

 Hijos $\leftarrow$ ordenar_y_eliminar_peores(Hijos, Actual)

si no vacío?(Hijos) **entonces**

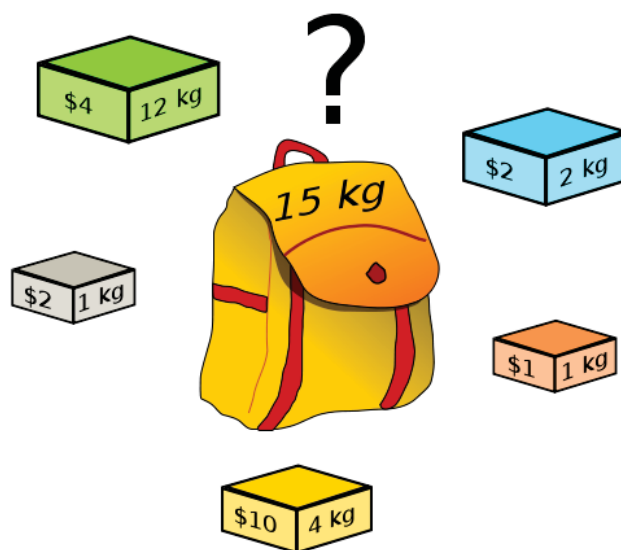
 Actual $\leftarrow$ Escoger_mejor(Hijos)

en otro caso

 fin $\leftarrow$ cierto

- ⦿ Sólo se consideran los descendientes cuya función de estimación es mejor que la del padre (poda del espacio de búsqueda)
- ⦿ Se puede usar una pila y guardar los hijos mejores que el padre para hacer backtracking, pero por lo general es prohibitivo
- ⦿ Es posible que el algoritmo no encuentre una solución aunque la haya

8

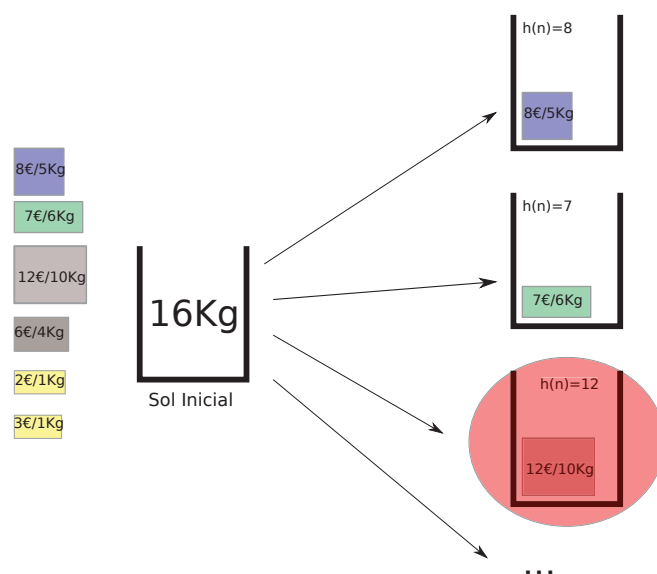


9

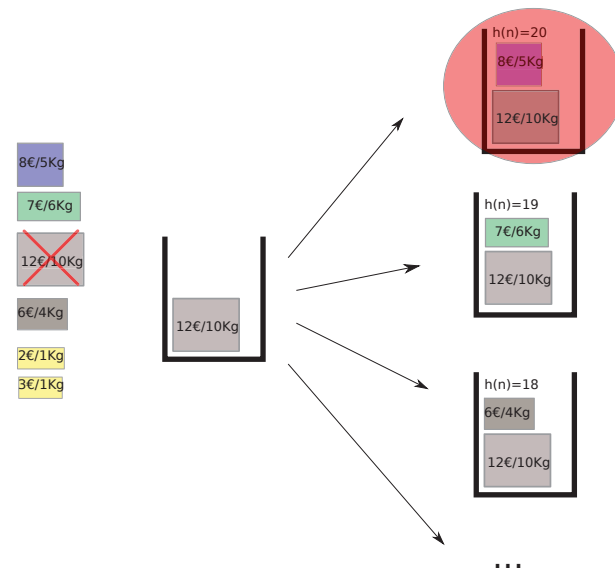
Problema de la Mochila

- ⊙ Solución: Cualquier combinación de objetos en la mochila que maximice valor y no supere el peso máximo
- ⊙ Solución Inicial:
 - opción 1: Mochila vacía
 - opción 2: Cualquier combinación de objetos generada al azar que no supere el peso máximo
- ⊙ Operadores:
 - Meter objeto en mochila [máximo N sucesores]
 - Sacar objeto de mochila [máximo N sucesores]
- ⊙ Función heurística:
 - opción 1: $\max \sum_i Valor_i$
 - opción 2: $\max \sum_i \frac{Valor_i}{Peso_i}$

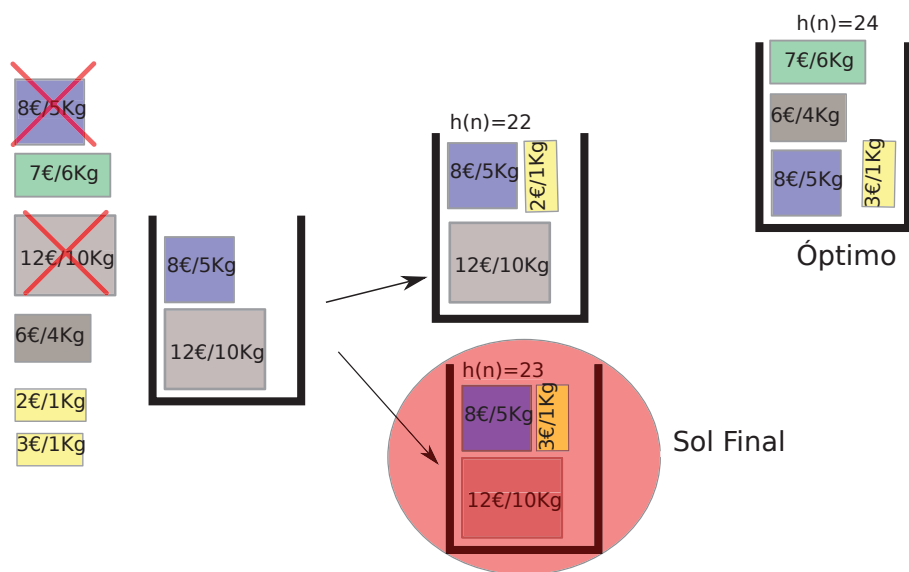
10



11



12



13

Problema del N viajante de comercio

- ⊙ Solución: Cada uno de los N camiones tiene una ruta asignada que empieza en el origen, pasa por todos los puntos de entrega asignados, y vuelve al origen. Por cada punto de entrega pasa un solo camión, una sola vez. La distancia recorrida por los camiones ha de minimizarse.
- ⊙ Solución Inicial: Asignamos al azar puntos de entrega a la ruta de cada camión, se establece un orden de visita al azar para cada ruta.
- ⊙ Operadores:
 - Mover un punto de entrega P_i de la ruta del camión C_j a la del camión C_k [(P*C) sucesores]
 - Swap entre dos puntos de entrega P_i y P_j de la ruta de un mismo camión C_k [máximo P*P sucesores]
- ⊙ Función heurística: suma de las distancias recorridas por los camiones en sus rutas (minimizar)

14

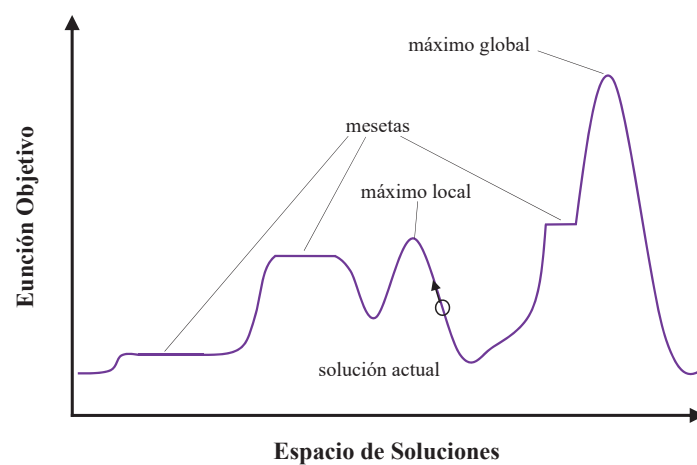
Problema de las N reinas

- ⊙ En este problema empezamos en el espacio de no soluciones (¡Es muy difícil generar una solución! Queremos que el algoritmo la encuentre)
- ⊙ Solución: Tablero en el que N reinas no se matan entre sí
- ⊙ Solución Inicial:
 - Cualquier tablero
 - Fijar a cada reina una columna, asignar a cada reina una fila al azar
- ⊙ Operadores:
 - Mover una reina en su columna [(N*N) - N sucesores]
 - Mover una reina a una casilla contigua en su columna [máximo 2*N sucesores]
- ⊙ Función heurística: número de parejas de reinas que se matan entre sí (minimizar)
- ⊙ ¿Otras maneras de plantear el problema?

15

- ⊙ Las características de la función heurística y la solución inicial determinan el éxito y rapidez de la búsqueda
- ⊙ La estrategia del algoritmo hace que la búsqueda pueda acabar en un punto donde la solución sólo sea la óptima aparentemente
- ⊙ Problemas
 - Máximo local: Ningún vecino tiene mejor coste
 - Meseta: Todos los vecinos son iguales
 - Cresta: La pendiente de la función sube y baja (efecto escalón)

16



17

⊙ Posibles soluciones

- Reiniciar la búsqueda en otro punto buscando mejorar la solución actual (*Random Restarting Hill Climbing*)
- Hacer backtracking a un nodo anterior y seguir el proceso en otra dirección (solo posible limitando la memoria para hacer el backtracking, *Beam Search*)
- Aplicar dos o más operaciones antes de decidir el camino
- Hacer HC en paralelo (p.ej. Dividir el espacio de búsqueda en regiones y explorar las más prometedoras, posiblemente compartiendo información)

- ⊙ Se han planteado otros algoritmos inspirados en analogías físicas y biológicas:
 - **Simulated annealing:** Hill-climbing estocástico inspirado en el proceso de enfriamiento de metales
 - **Algoritmos genéticos:** Hill-climbing paralelo inspirado en los mecanismos de selección natural
 - Ambos mecanismos se aplican a problemas reales con bastante éxito
- ⊙ Pero también Particle Swarm Optimization, Ant Colony Optimization, Intelligent Water Drop, Gravitational search algorithm, ...